

Roger Frazier, Alex Caramanico, Dereck Roman Rosario, John Kallis

Due Date: 3/10/2023

Group 1 MATH 341

Homework 4

“=====

Problem 1

Approximate second derivative $f''(x)$ of function $f(x) = e^x$ at $x = 1$ by second-order accurate centered approximation $D_h^{(2)}f(x)$. For $h = 1/2, 1/4, 1/8$, also compute their errors and the ratio $R_h(x) = E_{2h}^{(2)}(x) / E_h^{(2)}(x)$.

h	$D_h^{(2)}f(x)$	$E_h^{(2)}(x)$	$R_h(x)$
1/2	2.7753	-0.05716	N/A
1/4	2.7324	-0.01418	4.0268
1/8	2.7218	-0.00354	4.0062

Ideally, the ratio $R_h(x) \rightarrow 4$, and error $E_h^{(2)}(x) \rightarrow 0$ as $h \rightarrow 0$. In other words, the approximation of $f''(x)$ where $f(x) = e^x$ at $x = 1$ is becoming more accurate for smaller values of h . That is, with a smaller h , $D_h^{(2)}f(x) \rightarrow f''(1) = e^1 = 2.718$. A perfect approximation would have ratio $R_h(x) = 4$, which can be obtained in this instance because $f(x) = f^{(i)}(x)$ for all $i = 1, 2, \dots$. This ratio is 4 because h is being halved and then squared in the denominator. That is, for $h = 1/2$, the denominator of $R_h(x)$ contains $(1/2)^2 = 1/4$, all other components cancel out or are equivalent to 1, leaving a result of 4. This holds for all $h = 1/2^i$ for all $i = 1, 2, \dots$.

“=====

“=====”

Problem 2

Once again the intention here is to approximate the second derivative $f''(x)$ of function $f(x) = e^x$ at $x = 1$ by second-order accurate centered approximation $D_h^{(2)}f(x)$, based on numerical differentiation principles. In the first problem we used the approximation formulas derived in class for three separate values of h , but the following portions of the document describe an approach using an optimal h , along with several other values of h and explanations of the obtained results.

Using the formula derived from the lecture, the approximation is optimized at:

$$h_{opt} = \left(\frac{48\varepsilon}{M}\right)^{1/4}, \text{ where } M = \max|f^{(4)}(x)|, x = 1$$

Since $f^{(4)}(x) = f(x) = f(1) = e$, then $M = \max|f^{(4)}(x)| = e$, Euler's constant. Using $\varepsilon = 2.22 \cdot 10^{-16}$, h_{opt} can be calculated. This value was selected for ε because it is sufficiently small, ensures double-precision, and for ease of programming since $\varepsilon = eps$ is a predefined value in Matlab/Octave. Thus, h_{opt} was calculated to be 0.00025023, or $2.5023e - 04$.

For clarity, double-precision essentially means the computer detects a much smaller number as different to 0. That is, the smallest number recognized as *not 0* is 10^{-16} . The “doubling” nature essentially comes from the fact that there is an h^2 term in the approximation formulas. This will hypothetically lead to smaller errors. Additionally, double-precision will give 16 significant decimal digits.

The first column in the following table represents the step number of the h being used in the following approximation. The following columns are represented to 16 significant decimal digits.

The second represents the actual, finite value of h . The third is the approximation $D_h^{(2)}f(x)$.

Column four is the error $E_h^{(2)}(x) = f''(x) - D_h^{(2)}f(x)$, with the final column computing the

ratio between errors. That is, $R = R_h(x) = \frac{E_{2h}^{(2)}(x)}{E_h^{(2)}(x)}$.

Repeating these calculations for $h = 1/2^k$ for $k = 1, 2, 3, \dots, 20$, the resulting values were

k	h	D	E	R
1.000000	5.00000000000000e-01	2.7753867364804101e+00	-5.71049e-02	
2.000000	2.50000000000000e-01	2.7324690745028164e+00	-1.41872e-02	4.0250875924179850e+00
3.000000	1.25000000000000e-01	2.7218231018904930e+00	-3.54127e-03	4.0062554666868726e+00
4.000000	6.25000000000000e-02	2.7191668010490275e+00	-8.84973e-04	4.0015628410798909e+00
5.000000	3.12500000000000e-02	2.7185030500022549e+00	-2.21222e-04	4.0003906362010015e+00
6.000000	1.56250000000000e-02	2.7183371324954351e+00	-5.53040e-05	4.0000975995647865e+00
7.000000	7.81250000000000e-03	2.7182956543838372e+00	-1.38259e-05	4.0000243905103652e+00
8.000000	3.90625000000000e-03	2.7182852849364281e+00	-3.45648e-06	4.0000044149681386e+00
9.000000	1.95312500000000e-03	2.7182826926000416e+00	-8.64141e-07	3.9998997812053503e+00
10.000000	9.76562500000000e-04	2.7182820448651910e+00	-2.16406e-07	3.9931444320821465e+00
11.000000	4.88281250000000e-04	2.7182818837463856e+00	-5.52873e-08	3.9142079181115688e+00
12.000000	2.44140625000000e-04	2.7182818502187729e+00	-2.17597e-08	2.5408102986555665e+00
13.000000	1.22070312500000e-04	2.7182818353176117e+00	-6.85857e-09	3.1726349039398105e+00
14.000000	6.10351562500000e-05	2.7182818651199341e+00	-3.66609e-08	1.8708129541484891e-01
15.000000	3.05175781250000e-05	2.7182822227478027e+00	-3.94289e-07	9.2979797877918616e-02
16.000000	1.52587890625000e-05	2.7182846069335938e+00	-2.77847e-06	1.4190835681177988e-01
17.000000	7.62939453125000e-06	2.7182846069335938e+00	-2.77847e-06	1.0000000000000000e+00
18.000000	3.81469726562500e-06	2.7183227539062500e+00	-4.09254e-05	6.7891122478092633e-02
19.000000	1.90734863281250e-06	2.7183837890625000e+00	-1.01961e-04	4.0138490572006047e-01
20.000000	9.5367431640625e-07	2.7187500000000000e+00	-4.68172e-04	2.1778471037975650e-01

In octave we used the following code to obtain the table. The *if* statements are present so that the entire table can be output within the for loop, and to avoid a ratio being calculated for the first value of *h*.

```

1| f = @(x) (exp(x)); # define function f(x)
2| x = 1; # given x value
3|
4| disp(' k      h      D      E      R') # large white-space to format output
5| format1 = '%24f %24.13e %24.16e %12.5e %24.16e \n';
6| format2 = '%24f %24.13e %24.16e %12.5e \n';
7| error_vector = zeros(20,1); # empty vector to be filled with errors at each h, use to calculate R
8| h_opt = (48*eps/e)^(1/4); # optimal value of h using predefined value of epsilon
9| for k = 1:20 # 20 values of h
10| h = 1/2^k; # halving h
11| D = (f(x-h)-2*f(x)+f(x+h))/h^2; # approximation
12| E = f(x)-D; # error
13| error_vector(k) = E; # update error vector
14| if k == 1
15| fprintf(format2,k,h,D,E) # print row without ratio, since we don't have an above row
16| end
17| if k >= 2
18| R = error_vector(k-1) / error_vector(k); # calculate ratio, previous error / current error
19| fprintf(format1,k,h,D,E,R) # format all values into a "table", including the ratio
20| end
21| end

```

It can be seen from the above table that $R_h(x) \rightarrow 4$ as $h \rightarrow 0$, so long as h is greater than $h_{opt} = 0.00025023$, which occurs just before $k = 12$. Interestingly, the smallest error computed by the code occurred at $k = 13$. While this does not line up directly with h_{opt} , this slight imprecision is likely due to accumulated round-off occurring within the program. Further, the ratio seems to go awry around $k = 8$, essentially an indicator that the approximation is beginning to approach an optimal value for h . After this point, the approximation quality would diminish as errors begin to increase. We see that the errors begin to increase after $k = 13$, just after h_{opt} . Generally, the ratio $R_h(x)$ is an indicator of approximation quality which begins “misbehaving” sooner than the errors themselves. Though trivial based on the error values, it should also be noted that the approximation D approaches closer and closer to Euler’s constant as $h \rightarrow h_{opt}$, before beginning to diverge when $h < h_{opt}$.

In theory, the ratio $R_h(x) \rightarrow 4$ as $h \rightarrow 0$. This is a result of the theoretical convergence of numerical differentiation. That is, if h is repeatedly halved, it can be seen that the ratio between errors using $2h$ and h is always 4. This is shown by the following work.

Using the centered approximation to $f''(x)$ with second-order accuracy:

$$f''(x) = \frac{f(x-h) - 2f(x) + f(x+h)}{h^2} - \frac{1}{12}h^2 f^{(4)}(\xi)$$

$$E_h^{(2)}(x) := f''(x) - D_h^{(2)}f(x) = \frac{-1}{12}h^2 f^{(4)}(\xi),$$

with ξ as some intermediate value satisfying the approximation for $f''(x)$.

Now, with a given $f(x)$ and x value, along with a known $f''(x)$ as in this case with $f(x) = e^x$, it can be shown that:

$$|E_h^{(2)}(x)| = \frac{1}{12}h^2 |f^{(4)}(\xi)| \approx \frac{1}{12}h^2 |f^{(4)}(x)|, \text{ if } h \text{ is halved from } 2h, \text{ then it follows that}$$

$$|E_{2h}^{(2)}(x)| = \frac{1}{12}(2h)^2 |f^{(4)}(x)| = 4|E_h^{(2)}(x)|, \text{ or } = \frac{1}{3}h^2 |f^{(4)}(x)|, \text{ so}$$

$$R_h(x) = \frac{|E_{2h}^{(2)}(x)|}{|E_h^{(2)}(x)|} = \frac{\frac{1}{12}(2h)^2 |f^{(4)}(x)|}{\frac{1}{12}h^2 |f^{(4)}(x)|} = \frac{(2h)^2}{h^2} = \frac{4h^2}{h^2} = 4$$

This is to say that the error decreases by a factor of 4 at each successive value of h , specifically when h is halved at each step. If h were to be divided by 4 for each successive value instead of 2, the expected ratio would be 8. As mentioned, $R_h(x) \rightarrow 4$ as $h \rightarrow 0$, so long as h is greater than

$h_{opt} = 0.0025023$. In other words, the approximation $D_h^{(2)}f(x)$ converges to $f''(x)$ with second-order accuracy.

The stipulation that convergence only occurs when $h \geq h_{opt}$ is important to explore in more detail. This occurs because the value of h is squared in both sources of error for the approximation - one in the numerator, the other in the denominator. In numerical approximation, there is numerical error *and* accumulated roundoff, and in order to minimize both, the optimal value of h must be found. This is shown in the following derivation of a "perfect" approximation

$D_h^{(2)*}f(x)$. Using $f^*(x)$ to differentiate from $f(x)$:

$$\begin{aligned} D_h^{(2)*}f(x) &= \frac{f^*(x-h) - 2f^*(x) + f^*(x+h)}{h^2}, \text{ } f^* \text{ consists of } f(x) \text{ plus some errors } e_i \leq |\varepsilon| \\ &= \frac{f(x-h) + e_1 - 2(f(x) + e_2) + f(x+h) + e_3}{h^2} \\ &= \frac{f(x-h) + 2f(x) + f(x+h)}{h^2} + \frac{e_1 - 2e_2 + e_3}{h^2} \\ &= D_h^{(2)}f(x) + \frac{e_1 - 2e_2 + e_3}{h^2}, \text{ now substituting the approximation with its definition} \\ &= f''(x) + \frac{h^2}{12}f^{(4)}(\xi) + \frac{e_1 - 2e_2 + e_3}{h^2} \end{aligned}$$

From here it can be seen that the two sources of error are:

$$- \frac{h^2}{12}f^{(4)}(\xi), \text{ the numerical error, and}$$

$$\frac{e_1 - 2e_2 + e_3}{h^2}, \text{ the accumulated round-off}$$

Plotting both of these against h reveals that numerical error $\rightarrow 0$ as $h \rightarrow 0$, but accumulated round-off $\rightarrow \infty$ as $h \rightarrow 0$. This suggests that there is some intermediate value of h which minimizes *both* of these sources. In this problem, it is clear that the accumulated round-off increases when h is smaller than h_{opt} . Mathematically, this occurs because the smaller and smaller value in the denominator of the equation results in a larger overall value when the numerator is held constant. Formally:

$$\lim_{x \rightarrow 0^+} \frac{a}{x} = \infty, \text{ for any constant } a.$$

The division by h in the round-off term is inevitable, as the definition of a derivative involves this division, and since we are approximating derivatives, such a result cannot be avoided.

In our problem, each e_i is less than or equal to $|\varepsilon| = 2.22 \cdot 10^{-16}$ and fixed, so round-off increases as $h \rightarrow 0$, increasing the overall error when $h < h_{opt}$.

“=====”